

P0859R0: Core Issue 1581: When are constexpr member functions defined?

This paper provides wording for Core Issue 1581.

Change in basic.def.odr 6.2 paragraph 3, and **move the indicated portion to a new paragraph prior to paragraph 3**:

[...] A virtual member function is odr-used if it is not pure. **A function is odr-used if it is named by a potentially-evaluated expression.**

Begin text to be moved to new paragraph

A function is named by an expression as follows:

- A function whose name appears **in an expression is named by that expression** ~~as a potentially-evaluated expression is odr-used~~ if it is the unique lookup result or the selected member of a set of overloaded functions (6.4, 16.3, 16.4), unless it is a pure virtual function and either its name is not explicitly qualified or the expression forms a pointer to member (8.3.1). [*Note*: This covers **taking the address of functions ([conv.func], [expr.unary.op])**, calls to named functions (8.2.2), operator overloading (Clause 16), user-defined conversions (15.3.2), allocation functions for placement new-expressions (8.3.4), as well as non-default initialization (11.6). A constructor selected to copy or move an object of class type is **considered to be named by an expression** ~~odr-used~~ even if the call is actually elided by the implementation (15.8). — *end note*]
- An allocation or deallocation function for a class is **named** ~~odr-used~~ by a new-expression ~~appearing in a potentially-evaluated expression~~ as specified in 8.3.4 and 15.5.
- A deallocation function for a class is **named** ~~odr-used~~ by a delete expression ~~appearing in a potentially-evaluated expression~~ as specified in 8.3.5 and 15.5.

End text to be moved to new paragraph

A non-placement allocation or deallocation function for a class is odr-used by the definition of a constructor of that class. A non-placement deallocation function for a class is odr-used by the definition of the destructor of that class, or by being selected by the lookup at the point of definition of a virtual destructor (15.4). An assignment operator function in a class is odr-used by an implicitly-defined copy-assignment or move-assignment function for another class as specified in 15.8. A constructor for a class is odr-used as specified in 11.6. A destructor for a class is odr-used if it is potentially invoked (15.4).

Append a paragraph to expr.const 6.2:

An expression is *potentially constant evaluated* if it is:

- a potentially-evaluated expression ([basic.def.odr]),
- a *constraint-expression*, including one formed from the *constraint-logical-or-expression* of a *requires-clause*,

- an immediate subexpression of a *braced-init-list* [*Footnote*: Constant evaluation may be necessary to determine whether a narrowing conversion is performed ([*dcl.init.list*]).],
- an expression of the form *& cast-expression* that occurs within a templated entity [*Footnote*: Constant evaluation may be necessary to determine whether such an expression is value-dependent ([*temp.dep.constexpr*]).], or
- a subexpression of one of the above that is not a subexpression of a nested unevaluated operand.

A function or variable is *needed for constant evaluation* if it is:

- a *constexpr* function that is named by an expression ([*basic.def.odr*]) that is potentially constant evaluated, or
- a variable whose name appears as a potentially constant evaluated expression that is either a *constexpr* variable or is of non-volatile *const-qualified* integral type or of reference type.

Change in special 15 paragraph 1:

[Note: The implementation will implicitly declare these member functions for some class types when the program does not explicitly declare them. The implementation will implicitly define them if they are odr-used (6.2) **or needed for constant evaluation ([*expr.const*])**. See 15.1, 15.4 and 15.8. — end note] [...]

Change in class.ctor 15.1 paragraph 7:

A default constructor that is defaulted and not defined as deleted is implicitly defined when it is odr-used (6.2) to create an object of its class type (4.5), **when it is needed for constant evaluation ([*expr.const*])**, or when it is explicitly defaulted after its first declaration.

No change in class.dtor 15.4 paragraph 7:

A destructor that is defaulted and not defined as deleted is implicitly defined when it is odr-used (6.2) or when it is explicitly defaulted after its first declaration.

Change in class.copy.ctor 15.8.1 paragraph 12:

A copy/move constructor that is defaulted and not defined as deleted is implicitly defined ~~if~~ **when** it is odr-used (6.2), **when it is needed for constant evaluation ([*expr.const*])**, or when it is explicitly defaulted after its first declaration.

Change in class.copy.assign 15.8.2 paragraph 10:

A copy/move assignment operator for a class X that is defaulted and not defined as deleted is implicitly defined when it is odr-used (6.2) (e.g., when it is selected by overload resolution to assign to an object of its class type), **when it is needed for constant evaluation ([*expr.const*])**, or when it is explicitly defaulted after its first declaration.

Change in 17.8.1 temp.inst paragraph 3:

Unless a member of a class template or a member template has been explicitly instantiated or explicitly specialized, the specialization of the member is implicitly instantiated when the specialization is referenced in a context that requires the member definition to exist **or if the existence of the definition of the member affects the semantics of the program**; [...]

Change in 17.8.1 temp.inst paragraph 4:

Unless a function template specialization has been explicitly instantiated or explicitly specialized, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist **or if the existence of the definition affects the semantics of the program**. A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context that requires a function definition to exist **or if the existence of the definition affects the semantics of the program**. [...]

Change in 17.8.1 temp.inst paragraph 6:

Unless a variable template specialization has been explicitly instantiated or explicitly specialized, the variable template specialization is implicitly instantiated when ~~the specialization is used~~ **it is referenced in a context that requires a variable definition to exist or if the existence of the definition affects the semantics of the program**. A default template argument for a variable template is implicitly instantiated when the variable template is referenced in a context that requires the value of the default argument.

Add a new paragraph after 17.8.1 temp.inst paragraph 6:

The existence of a definition of a variable or function is considered to affect the semantics of the program if the variable or function is needed for constant evaluation by an expression, even if constant evaluation of the expression is not required or if constant expression evaluation does not use the definition. [*Example*:

```
template<typename T> constexpr int f() { return T::value; }
template<bool B, typename T> void g(decltype(B ? f<T>() : 0));
template<bool B, typename T> void g(...);
template<bool B, typename T> void h(decltype(int{B ? f<T>() : 0}));
template<bool B, typename T> void h(...);
void x() {
    g<false, int>(0); // OK, B ? f<T>() : 0 is not potentially constant evaluated
    h<false, int>(0); // error, instantiates f<int> even though B evaluates to false and
                    // list-initialization of int from int cannot be narrowing
}
]
```